

IntellMeet — Backend Workstream Audit

Member 2 (Backend, Database & Real-Time Infrastructure) • Week 1 Deliverables vs Actual Progress

PROJECT	TEAM MEMBER	TARGET MILESTONE	AUDIT DATE
IntellMeet (MERN + AI)	Member 2 (Backend Lead)	Week 1 (Days 1 to 7)	March 2026 (Live Audit)

WEEK 1 COMPLETION

78%

Core architecture & APIs operational

AHEAD-OF-SCHEDULE

4 Features

WebRTC, Live Chat, Tasks & Admin

PENDING WEEK 1 GAPS

4 Critical

Refresh Token, Redis, Cloudinary, Delete

CODE HEALTH

Production

TypeScript + MongoMemoryServer ready

DAY-BY-DAY EXECUTION PLAN VS ACTUAL IMPLEMENTATION

DAY	PLANNED MILESTONE (28-DAY PLAN)	ACTUAL IMPLEMENTED CODEBASE STATUS	STATUS
Day 1	MERN boilerplate with Vite & Express; MongoDB setup; install dependencies; initial commit.	Express + TS server, Helmet, CORS, Rate Limit in <code>server.ts</code> . Mongoose connection with Memory-Server fallback in <code>config/db.ts</code> .	✓ Completed
Day 2	User model & auth routes (signup, login); JWT with refresh tokens; password hashing with bcrypt.	<code>User.ts</code> with bcrypt pre-save hashing & roles. Register, Login, Me in <code>authController.ts</code> . <i>*Refresh tokens not implemented.</i>	⚠ Partial (No Refresh)
Day 3	Profile creation & avatar upload via Cloudinary; protected routes with middleware; auth rate limiting.	JWT verify & role RBAC in <code>middleware/auth.ts</code> . Profile update working. Global rate limiting set. <i>*Cloudinary upload & auth rate limit missing.</i>	⚠ Partial (No Cloudinary)
Day 4	Meeting model and basic CRUD for meetings; WebRTC setup for video calls (peer connection logic).	<code>Meeting.ts</code> schema; Create, Get, RoomId lookup in <code>meetingController.ts</code> . WebRTC signaling in <code>socketHandler.ts</code> . <i>*Delete API missing.</i>	✓ Mostly Done (85%)
Day 5	Redis setup for session and meeting caching; Socket.io server configuration for real-time features.	Socket.io server configured with room joins, participant roster, audio/video toggle, screen sharing. <i>*Redis cache completely missing (in-memory Map used).</i>	⚠ Partial (No Redis)
Day 6	Basic chat functionality in meetings; Real-time notification setup using Socket.io events.	<code>Message.ts</code> chat persistence, typing indicators & system logs working. <code>Notification.ts</code> REST API ready. <i>*Socket notification push missing.</i>	✓ Mostly Done (80%)
Day 7	Week 1 Checkpoint: Backend running locally, auth working, meeting creation, Postman test, README.	Local TS server operational with automated seeding (<code>seedData.ts</code>). Detailed <code>README.md</code> present. <i>*git repo & Postman file missing.</i>	✓ Completed



What Member 2 Did Exceptionally Well



Critical Gaps That Must Be Resolved

- ✓ **Rock-solid Foundation:** Express 4 + TypeScript setup with strict typing, central error handling, and robust CORS/Helmet security.
- ✓ **Resilient Database Fallback:** Added `mongodb-memory-server` so teammates can run the server instantly even without local MongoDB installed.
- ✓ **Early Delivery of Week 2 WebRTC:** Implemented `webrtc:offer`, `webrtc:answer`, and `webrtc:ice-candidate` a whole week ahead of schedule.
- ✓ **Early Delivery of Week 3 Modules:** Full Task Management CRUD (`taskController.ts`), Notification CRUD, and Admin Audit Logging already implemented.

- ✗ **Missing Refresh Tokens:** Day 2 specified JWT with refresh tokens. Currently only a 7-day single token is issued without refresh rotation or cookie storage.
- ✗ **Missing Cloudinary Integration:** Day 3 specified avatar upload via Cloudinary. No Multer or Cloudinary SDK exists; avatars rely solely on URL strings.
- ✗ **Missing Redis Session Cache:** Day 5 specified Redis setup for active meeting rooms. Server currently tracks users in in-memory JavaScript Maps.
- ✗ **Missing Meeting Delete API:** CRUD is incomplete without `DELETE /api/meetings/:id` in `meetingController.ts`.

DEEP-DIVE TECHNICAL GAP ANALYSIS & ACTION PLAN

MISSING FEATURE	PRIORITY	REQUIREMENT VS CODEBASE REALITY	RECOMMENDED FIX FOR MEMBER 2
JWT Refresh Token Rotation	HIGH PRIORITY	Plan requires access token (short lived) + refresh token (long lived). Codebase only generates one token (<code>expiresIn: 7d</code>).	Create <code>POST /api/auth/refresh</code> endpoint. Store refresh token in HTTP-only cookie or DB model with invalidation logic.
Cloudinary Avatar Upload	HIGH PRIORITY	Plan requires media upload for user profiles. Server has neither <code>cloudinary</code> nor <code>multer</code> in <code>package.json</code> .	Install <code>cloudinary</code> & <code>multer</code> . Create <code>POST /api/users/avatar</code> route to handle multipart form file uploads.
Meeting Delete API (CRUD)	HIGH PRIORITY	CRUD definition requires Create, Read, Update, Delete. <code>meetingController.ts</code> has everything except delete.	Add <code>deleteMeeting</code> in <code>meetingController.ts</code> and bind <code>router.delete('/:id')</code> in <code>meetingRoutes.ts</code> .
Redis Session Caching	MEDIUM PRIORITY	Plan specifies Redis caching for meetings & sessions. Current code uses local <code>new Map()</code> which does not scale horizontally.	Install <code>ioredis</code> . Cache active room participants and temporary meeting state in Redis hashes or key-value stores.
Auth-Specific Rate Limiting	MEDIUM PRIORITY	Day 3 specifies rate limiting on auth routes to prevent brute-force attacks. Currently only a broad 1000 req/15min rule exists on <code>/api</code> .	Configure strict limiter (5-10 requests per 15 mins) specifically mounted on <code>/api/auth/login</code> and <code>/api/auth/register</code> .
Socket Notification Push	LOW PRIORITY	Day 6 specifies real-time notification setup. REST APIs exist, but notifications are not pushed over Socket.io to online clients.	Emit <code>notification:received</code> inside <code>socketHandler.ts</code> when tasks or meeting invites are triggered.

AHEAD-OF-SCHEDULE FEATURES DELIVERED BY MEMBER 2

FEATURE MODULE	ORIGINAL WEEK	WHAT HAS ALREADY BEEN IMPLEMENTED
WebRTC Peer Signalling	Week 2 (Backend)	Full low-latency signaling (<code>webrtc:offer</code> , <code>webrtc:answer</code> , <code>webrtc:ice-candidate</code>) implemented in <code>sockets/socketHandler.ts</code> . Ready for Frontend Member 1!
In-Meeting Real-time Chat	Week 2 (Backend)	Persistent chat system using <code>Message.ts</code> , room broadcasting, real-time typing indicators (<code>chat:typing</code>), and automatic arrival/departure system messages.
Task & Kanban Backend APIs	Week 3 (Backend)	Full Task CRUD (<code>Task.ts</code> , <code>taskController.ts</code>) with status transitions (to-do, in-progress, completed), priority levels, meeting linkage, and assignee population.
Admin Governance & Audit Logs	Week 3 (Backend)	Enterprise audit trail (<code>AuditLog.ts</code>) recording user registration, logins, meeting creation, and admin user status toggles in <code>adminController.ts</code> .

🚀 Executive Recommendation for Week 2 Handoff

Member 2 has delivered exceptional architectural groundwork. Because Member 2 has already completed the WebRTC signaling and Chat modules slated for Week 2, the team is in a **very strong position**. To achieve 100% compliance with Week 1 specifications before moving forward:

Immediate Step (1-2 Hours): Implement `deleteMeeting` in `meetingController.ts` and add auth-specific rate limiting.

Secondary Step (1 Day): Implement Refresh Tokens and Cloudinary/S3 image uploads so Member 1 (Frontend) can integrate genuine profile picture uploads during Week 2.

